



19. Python: Regular Expressions

"Some people, when confronted with a problem, think 'I know, I'll use regular expressions.' Now they have two problems."

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 18. Python: Lambda Functions](#)

Contents:

[1. RegEx Functions \(Cheatsheet\)](#)

[Common RegEx Functions](#)

[Special Characters](#)

[RegEx Flags](#)

[Special Sequences](#)

[Character Sets](#)

[Quick Reminder](#)

[3. Some Practical Examples](#)

[Retrieve only those with texts](#)

[Start with **a** and ends with **c**](#)

[Start with **a** and ends with one or more **b**](#)

[Has exactly three occurrences of **d**](#)

[Matches the exact character **"food"**](#)

[Find only uppercase letters](#)

[Find capitalized words](#)

[Has **"xyz"** in between but not at start or end](#)

[Starts with "Hello"](#)

[Ends with "Flower"](#)

[Two consecutive words both start with the letter P](#)

[All words starting with either a, e, or u](#)

[String contains uppercase, lowercase, numbers, and underscores only](#)

[Check if a string starts with a specific number](#)

[Convert date from yyyy-mm-dd to dd-mm-yyyy](#)

[Check if a string ends with a number \(also show the number\)](#)

[Extract all digit values in the text](#)

[Extract all unique words in the passage](#)

[Find occurrences of a specific substring](#)

[Replace "Hello" with "Goodbye"](#)

[Extract Email Addresses](#)

[Extract Date from URL](#)

[Summary](#)

[References](#)

Regular Expressions, or simply **Regex**, is one of those topics that looks simple at first: "Find patterns inside text."

And honestly, that *is* the core idea. But once we start using Regex seriously, we quickly realize something important: the syntax can become extremely powerful, and sometimes a little terrifying.

That's because Regular Expressions are designed to describe patterns in text with very compact symbols. A single pattern can search through thousands of words, validate user input, extract information, replace text, or even clean messy datasets automatically.

So instead of checking strings character by character manually, we let Regex handle the pattern matching for us.

In Python, we work with Regular Expressions using the built-in `re` module.

```
import re
```

Think of Regex as a special language for searching text. We can use it to:

- Find phone numbers
- Validate emails
- Extract hashtags
- Search logs
- Replace words
- Split text intelligently
- Clean datasets
- Detect patterns automatically

But here's the important part: Regex is powerful only when used in the right place. Sometimes a simple string method like `.replace()` or `.split()` is enough.

Using Regex for everything can actually make code harder to read. So throughout this section, we'll focus on understanding what Regex does, how the syntax works, and when it actually becomes useful.

1. RegEx Functions (Cheatsheet)

Common RegEx Functions

Function	Description
<code>findall()</code>	Returns a list containing all matches
<code>search()</code>	Returns a Match object if there is a match anywhere in the string
<code>split()</code>	Returns a list where the string has been split at each match
<code>sub()</code>	Replaces one or many matches with a string
<code>span()</code>	Returns the start and end positions of a match
<code>group()</code>	Returns the matched part of the string

Function	Description
<code>string()</code>	Returns the original string from the Match object

Special Characters

Character	Description	Example
<code>[]</code>	A set of characters	<code>"[a-m]"</code>
<code>\</code>	Signals a special sequence or escapes characters	<code>"\d"</code>
<code>.</code>	Any character except newline	<code>"he..o"</code>
<code>^</code>	Starts with	<code>"^hello"</code>
<code>\$</code>	Ends with	<code>"planet\$"</code>
<code>*</code>	Zero or more occurrences	<code>"he.*o"</code>
<code>+</code>	One or more occurrences	<code>"he.+o"</code>
<code>?</code>	Zero or one occurrence	<code>"he.?o"</code>
<code>{}</code>	Exactly the specified number of occurrences	<code>"he.{2}o"</code>
<code> </code>	Either/or	<code>"falls stays"</code>
<code>()</code>	Capture and group	<code>"(abc)"</code>

RegEx Flags

Flag	Shorthand	Description
<code>re.ASCII</code>	<code>re.A</code>	Returns only ASCII matches
<code>re.DEBUG</code>		Returns debug information
<code>re.DOTALL</code>	<code>re.S</code>	Makes <code>.</code> match all characters including newline
<code>re.IGNORECASE</code>	<code>re.I</code>	Case-insensitive matching
<code>re.MULTILINE</code>	<code>re.M</code>	Matches start/end of each line
<code>re.NOFLAG</code>		Specifies no flag
<code>re.UNICODE</code>	<code>re.U</code>	Returns Unicode matches

Flag	Shorthand	Description
<code>re.VERBOSE</code>	<code>re.X</code>	Allows spaces and comments inside patterns

Special Sequences

Sequence	Description	Example
<code>\A</code>	Match at beginning of string	<code>"\AThe"</code>
<code>\b</code>	Match at beginning/end of word	<code>r"\bain"</code>
<code>\B</code>	Match NOT at beginning/end of word	<code>r"\Bain"</code>
<code>\d</code>	Match digits (<code>0-9</code>)	<code>"\d"</code>
<code>\D</code>	Match non-digits	<code>"\D"</code>
<code>\s</code>	Match whitespace	<code>"\s"</code>
<code>\S</code>	Match non-whitespace	<code>"\S"</code>
<code>\w</code>	Match word characters	<code>"\w"</code>
<code>\W</code>	Match non-word characters	<code>"\W"</code>
<code>\Z</code>	Match at end of string	<code>"Spain\Z"</code>

Character Sets

Set	Description
<code>[arn]</code>	Match <code>a</code> , <code>r</code> , or <code>n</code>
<code>[a-n]</code>	Match lowercase letters from <code>a</code> to <code>n</code>
<code>[^arn]</code>	Match any character except <code>a</code> , <code>r</code> , or <code>n</code>
<code>[0123]</code>	Match <code>0</code> , <code>1</code> , <code>2</code> , or <code>3</code>
<code>[0-9]</code>	Match any digit from <code>0</code> to <code>9</code>
<code>[0-5][0-9]</code>	Match numbers from <code>00</code> to <code>59</code>
<code>[a-zA-Z]</code>	Match lowercase or uppercase letters
<code>[+]</code>	Match the <code>+</code> character literally

Quick Reminder

Regex patterns often use **raw strings** with `r""`.

```
pattern = r"\d+"
```

Why? Because raw strings prevent Python from treating `\` as an escape character before Regex sees it.

3. Some Practical Examples

Retrieve only those with texts

Example: `"Hello"` is valid, but `"Hello12"` is not.

```
import re

text = ["Hello", "Hello12", "World", "Python99"]

for word in text:
    if re.fullmatch(r"[A-Za-z]+", word):
        print(word)
```

Hello
World

Start with `a` and ends with `c`

```
import re

text = ["abc", "ac", "abbc", "xbc"]

for word in text:
    if re.fullmatch(r"a.*c", word):
        print(word)
```

abc
ac
abbc

Start with `a` and ends with one or more `b`

Example: `"a"` is invalid.

```
import re

text = ["ab", "abb", "abbbb", "a"]

for word in text:
    if re.fullmatch(r"ab+", word):
        print(word)
```

ab
abb
abbbb

Has exactly three occurrences of `d`

```
import re

text = ["ddd", "add", "dda", "ddd"]

for word in text:
    if re.fullmatch(r"^[^d]*d[^d]*d[^d]*d[^d]*", word) and word.count("d") == 3:
        print(word)
```

ddd
add
dda

Matches the exact character `"food"`

```
import re

text = "food is the thing that gathers people together. food is important."

# Match only if the string starts with "food"
match = re.match(r"food", text)

if match:
    print("Matched!")

# Count all occurrences of "food" -> make it case-insensitive using re.IGNORECASE
occurrences = re.findall(r"food", text, re.IGNORECASE)
print("Number of occurrences:", len(occurrences))
```

Matched!
Number of occurrences: 2

Find only uppercase letters

```
import re

text = "Hello WORLD PYTHON"

result = re.findall(r"[A-Z]", text)
print(result)
```

['H', 'W', 'O', 'R', 'L', 'D', 'P', 'Y', 'T', 'H', 'O', 'N']

Find capitalized words

First letter uppercase followed by lowercase letters

```
import re

text = "Python is Powerful and Regex is Amazing"

result = re.findall(r"\b[A-Z][a-z]+\b", text)
print(result)
```

['Python', 'Powerful', 'Regex', 'Amazing']

Has "xyz" in between but not at start or end

Example: "abxyzf" is valid, but "xyzabc" or "abcxyz" are invalid.

```
import re

text = ["abxyzf", "xyzabc", "abcxyz", "helloxyzworld"]

for word in text:
    if re.fullmatch(r".+xyz.+", word):
        print(word)
```

abxyzf
helloxyzworld

Starts with "Hello"

```
import re

text = "Hello World"

if re.match(r"^Hello", text):
    print("Starts with Hello")
```

Starts with Hello

Ends with "Flower"

```
import re

text = "Sun Flower"

if re.search(r"Flower$", text):
    print("Ends with Flower")
```

Ends with Flower

Two consecutive words both start with the letter **P**

```
import re

text = "Peter Parker protects people"

result = re.search(r"\bP\w+\s+P\w+\b", text)

if result:
    print(result.group())
```

Peter Parker

All words starting with either **a**, **e**, or **u**

```
import re

text = "apple banana umbrella elephant orange"

result = re.findall(r"\b[aeuAEU]\w*", text)
print(result)
```

['apple', 'umbrella', 'elephant']

String contains uppercase, lowercase, numbers, and underscores only

```
import re

text = "Hello_123"

if re.fullmatch(r"[A-Za-z0-9_]+", text):
    print("Valid")
```

Valid

Check if a string starts with a specific number

```
import re

text = "123Python"

if re.match(r"123", text):
    print("Starts with 123")
```

Starts with 123

Convert date from `yyyy-mm-dd` to `dd-mm-yyyy`

```
import re

date = "2026-05-27"

result = re.sub(r"(\d{4})-(\d{2})-(\d{2})", r"\3-\2-\1", date)
print(result)
```

27-05-2026

Check if a string ends with a number (also show the number)

```
import re

text = "Python99"

result = re.search(r"(\d+)$", text)

if result:
    print("Ends with number:", result.group(1))
```

Ends with number: 99

Extract all digit values in the text

```
import re

text = "I have 2 apples and 15 bananas"

result = re.findall(r"\d+", text)
print(result)
```

['2', '15']

Extract all unique words in the passage

```
import re

text = "Python is fun and Python is powerful"

words = re.findall(r"\b\w+\b", text)
unique_words = set(words)
print(unique_words)
```

{'Python', 'is', 'fun', 'and', 'powerful'}

Find occurrences of a specific substring

```
import re

text = "Python is fun. Python is powerful."

search_word = input("Enter word to find: ")
result = re.findall(search_word, text)
print("Occurrences:", len(result))
```

Enter word to find: Python

Occurrences: 2

Replace "Hello" with "Goodbye"

```
import re

text = "Hello World"

result = re.sub(r"Hello", "Goodbye", text)
print(result)
```

Goodbye World

Extract Email Addresses

```
import re

text = "Contact us at support@gmail.com or admin@yahoo.com"

emails = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", text)
print(emails)
```

```
['support@gmail.com', 'admin@yahoo.com']
```

Extract Date from URL

```
import re

url = "https://website.com/2026/05/27/article"

result = re.search(r"(\d{4})/(\d{2})/(\d{2})", url)

if result:
    print("Year:", result.group(1))
    print("Month:", result.group(2))
    print("Day:", result.group(3))
```

```
Year: 2026
```

```
Month: 05
```

```
Day: 27
```

Summary

Regex is one of the most powerful tools for working with text in Python.

Instead of manually checking characters one by one, we can describe patterns using a compact syntax and let the `re` module do the searching for us.

Throughout these examples, we used Regex to:

- Validate text formats

- Search for patterns
- Extract useful information
- Replace parts of strings
- Split and clean text
- Detect emails, dates, numbers, and words

And this is why Regex becomes so important in real-world programming. Whenever we need to process large amounts of text automatically—whether in web development, data science, cybersecurity, machine learning, or automation—Regex becomes an incredibly useful tool.

But at the same time, Regex should be used carefully. Simple string methods are often easier to read and maintain. Regex shines when pattern matching becomes too complex for normal string operations.

So the real skill is not just learning Regex syntax, it is knowing when Regex is the right tool for the job.

References

<https://www.geeksforgeeks.org/python/regular-expression-python-examples/>

<https://pynative.com/python-regex-exercises/>

<https://developers.google.com/edu/python/regular-expressions>

Next Section:

</> [20. Python: File Handling](#)